

第三次次上机作业：

学号：241830137 姓名：朱子健

2026 年 4 月 15 日

1 问题

考虑函数 $R(x) = \frac{1}{1+x^2}$. 利用下列条件做插值逼近，并且与 $R(x)$ 的图像比较.

- (1) 用节点 $x_i = 5\cos(\frac{2i+1}{42}\pi), i = 0, 1, \dots, 20$.
画出 20 次 Lagrange 插值多项式的图像；
- (2) 用等距节点 $x_i = -5 + i, i = 0, 1, \dots, 10$.
画出 10 次 Newton 插值多项式的图像；
- (3) 用等距节点 $x_i = -5 + i, i = 0, 1, \dots, 10$.
画出分段线性插值函数的图像；

2 算法原理与设计

2.1 拉格朗日插值法 (Lagrange Interpolation)

2.1.1 算法原理

拉格朗日插值法的核心是通过构造一组基函数 $l_i(x)$ 来形成插值多项式。对于给定 $n+1$ 个互异节点 $(x_i, y_i), i = 0, 1, \dots, n$, 插值多项式 $L_n(x)$ 满足 $L_n(x_i) = y_i$ 。

1. 拉格朗日基函数：定义为 n 次多项式，满足在节点 x_i 处为 1，在其他节点处为 0。

$$l_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j} = \frac{(x - x_0) \dots (x - x_{i-1})(x - x_{i+1}) \dots (x - x_n)}{(x_i - x_0) \dots (x_i - x_{i-1})(x_i - x_{i+1}) \dots (x_i - x_n)} \quad (1)$$

2. 插值多项式形式：

$$L_n(x) = \sum_{i=0}^n y_i l_i(x) \quad (2)$$

2.1.2 算法设计

算法 1 计算拉格朗日插值多项式

```
1: 输入: 节点数组  $X = [x_0, \dots, x_n]$ , 函数值  $Y = [y_0, \dots, y_n]$ , 待计算点  $x$ 
2: 输出: 插值结果  $P$ 
3:  $P \leftarrow 0$ 
4: for  $i = 0$  to  $n$  do
5:    $L \leftarrow 1$ 
6:   for  $j = 0$  to  $n$  do
7:     if  $j \neq i$  then
8:        $L \leftarrow L \times \frac{x - x_j}{x_i - x_j}$ 
9:     end if
10:  end for
11:   $P \leftarrow P + y_i \times L$ 
12: end for
13: return  $P$ 
```

2.2 牛顿插值法 (Newton Interpolation)

2.2.1 算法原理

牛顿插值法利用差商 (Divided Difference) 构造插值多项式。其优点是具有承袭性, 即增加一个节点时, 只需在原有多项式基础上增加一项。

1. 差商定义:

- 零阶差商: $f[x_i] = f(x_i)$
- 一阶差商: $f[x_i, x_j] = \frac{f(x_j) - f(x_i)}{x_j - x_i}$
- k 阶差商: $f[x_0, x_1, \dots, x_k] = \frac{f[x_1, \dots, x_k] - f[x_0, \dots, x_{k-1}]}{x_k - x_0}$

2. 牛顿插值多项式:

$$N_n(x) = f(x_0) + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots + f[x_0, \dots, x_n]\omega_n(x) \quad (3)$$

其中 $\omega_n(x) = \prod_{j=0}^{n-1} (x - x_j)$ 。

2.2.2 算法设计

算法 2 计算牛顿插值多项式

```
1: 输入: 节点  $x_0, \dots, x_n$ , 函数值  $f(x_0), \dots, f(x_n)$ , 点  $x$ 
2: 第一步 (计算差商表):
3: 令  $F_{i,0} = f(x_i)$  (差商表第一列)
4: for  $j = 1$  to  $n$  do
5:   for  $i = j$  to  $n$  do
6:      $F_{i,j} \leftarrow \frac{F_{i,j-1} - F_{i-1,j-1}}{x_i - x_{i-j}}$ 
7:   end for
8: end for
9: 第二步 (利用 Horner 算法求值):
10:  $y \leftarrow F_{n,n}$ 
11: for  $k = n - 1$  downto  $0$  do
12:    $y \leftarrow F_{k,k} + (x - x_k) \times y$ 
13: end for
14: return  $y$ 
```

2.3 分段线性插值 (Piecewise Linear Interpolation)

2.3.1 算法原理

分段线性插值是在每两个相邻节点 $[x_i, x_{i+1}]$ 之间利用线性函数来逼近原函数。这避免了高次多项式插值可能产生的 Runge 现象。

1. 定义: 在区间 $[a, b]$ 上, 设 $a = x_1 < x_2 < \dots < x_{n+1} = b$ 。在每个子区间 $[x_i, x_{i+1}]$ 上, 插值函数 $g_1(x)$ 为线性多项式。

2. 局部表达式: 当 $x \in [x_i, x_{i+1}]$ 时:

$$g_{1,i}(x) = f(x_i) + \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}(x - x_i) \quad (4)$$

3. 误差界:

$$|f(x) - g_1(x)| \leq \frac{h^2}{8} \max |f''(x)|, \quad h = \max(x_{i+1} - x_i) \quad (5)$$

2.3.2 算法设计

算法 3 计算分段线性插值函数

```
1: 输入: 已排序节点数组  $X$ , 函数值数组  $Y$ , 待求点  $x$ 
2: 第一步 (定位区间):
3: 在  $X$  中查找  $i$ , 使得  $x_i \leq x \leq x_{i+1}$  (可使用二分查找)
4: 第二步 (线性计算):
5:  $k \leftarrow \frac{y_{i+1} - y_i}{x_{i+1} - x_i}$ 
6:  $y \leftarrow y_i + k \times (x - x_i)$ 
7: return  $y$ 
```

3 结果分析

根据题目给定的节点和原函数 $R(x) = \frac{1}{1+x^2}$ ，我们可以写出这三种插值函数的具体数学表达式。

3.1 20 次 Lagrange 插值多项式

所用节点为：

$$x_i = 5 \cos \left(\frac{2i+1}{42} \pi \right), \quad i = 0, 1, \dots, 20$$

对应的函数值为 $y_i = \frac{1}{1+x_i^2}$ 。则具体的 20 次拉格朗日插值多项式表达式为：

$$L_{20}(x) = \sum_{i=0}^{20} \left(\frac{1}{1 + 25 \cos^2 \left(\frac{2i+1}{42} \pi \right)} \cdot \prod_{\substack{j=0 \\ j \neq i}}^{20} \frac{x - 5 \cos \left(\frac{2j+1}{42} \pi \right)}{5 \cos \left(\frac{2i+1}{42} \pi \right) - 5 \cos \left(\frac{2j+1}{42} \pi \right)} \right) \quad (6)$$

3.2 10 次 Newton 插值多项式

所用等距节点为 $\{-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5\}$ 。10 次牛顿插值多项式的具体展开式为：

$$\begin{aligned} N_{10}(x) = & c_0 + c_1(x+5) + c_2(x+5)(x+4) + c_3(x+5)(x+4)(x+3) \\ & + c_4(x+5)(x+4)(x+3)(x+2) + \dots \\ & + c_{10}(x+5)(x+4)(x+3)(x+2)(x+1)x(x-1)(x-2)(x-3)(x-4) \end{aligned} \quad (7)$$

其中，各项差商系数 $c_k = f[x_0, x_1, \dots, x_k]$ 经计算（保留五位有效小数）如下：

$$\begin{aligned} c_0 &\approx 0.03846 & c_1 &\approx 0.02036 & c_2 &\approx 0.01041 & c_3 &\approx 0.00633 \\ c_4 &\approx 0.00430 & c_5 &\approx -0.00204 & c_6 &\approx -0.00113 & c_7 &\approx 0.00109 \\ c_8 &\approx -0.00043 & c_9 &\approx 0.00011 & c_{10} &\approx -0.00002 \end{aligned}$$

3.3 分段线性插值多项式

等距节点同样为 $x_i \in \{-5, -4, \dots, 5\}$ ，函数值 $y_i \in \{\frac{1}{26}, \frac{1}{17}, \frac{1}{10}, \frac{1}{5}, \frac{1}{2}, 1, \dots\}$ 。通过计算两点间的斜率，我们可以写出精确的 10 段线性函数表达式：

$$P(x) = \begin{cases} \frac{1}{26} + \frac{9}{442}(x+5), & x \in [-5, -4] \\ \frac{1}{17} + \frac{7}{170}(x+4), & x \in [-4, -3] \\ \frac{1}{10} + \frac{1}{10}(x+3), & x \in [-3, -2] \\ \frac{1}{5} + \frac{3}{10}(x+2), & x \in [-2, -1] \\ \frac{1}{2} + \frac{1}{2}(x+1), & x \in [-1, 0] \\ 1 - \frac{1}{2}x, & x \in [0, 1] \\ \frac{1}{2} - \frac{3}{10}(x-1), & x \in [1, 2] \\ \frac{1}{5} - \frac{1}{10}(x-2), & x \in [2, 3] \\ \frac{1}{10} - \frac{7}{170}(x-3), & x \in [3, 4] \\ \frac{1}{17} - \frac{9}{442}(x-4), & x \in [4, 5] \end{cases} \quad (8)$$

3.4 图像对比

通过编写 Python 程序计算并绘制插值曲线，我们可以得出如下分析结论：

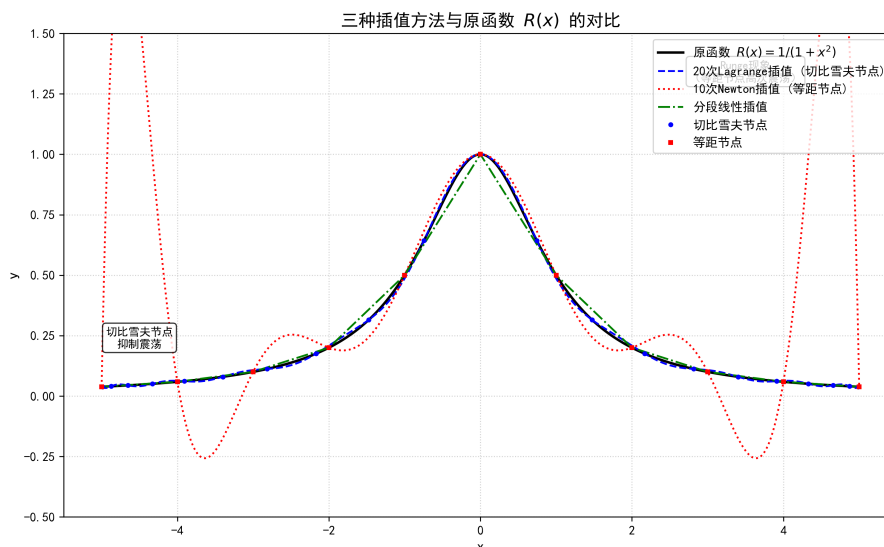


图 1: 三种插值方法与原函数 $R(x)$ 的对比示意图

1. **Runge 现象（等距节点高次插值）：**观察 10 次牛顿插值多项式的图像，在区间中心（0 附近）逼近效果较好，但在区间边缘（ $[-5, -3]$ 和 $[3, 5]$ ）出现了极其剧烈的震荡。这正是经典的 **Runge 现象**，说明对于 $R(x) = \frac{1}{1+x^2}$ 这样的函数，盲目增加等距节点的插值多项式次数，不仅无法提高精度，反而会导致边界处的误差发散。
2. **切比雪夫节点的优化效果：**在第 (1) 题中，使用了 20 次拉格朗日插值，但节点选取了切比雪夫零点（即两端密、中间疏的节点分布）。图像显示，尽管次数高达 20 次，不仅没有出现 Runge 现象，反而极其精确地贴合了原函数曲线。这验证了改变节点分布是克服 Runge 现象的有效手段。
3. **分段线性插值的稳健性：**第 (3) 题的分段线性插值在整个区间内表现出高度的稳定性。虽然它在节点处的导数不连续（表现为折线），逼近精度受限于节点数量，但它将误差严格限制在局部子区间内，完全避免了全局多项式的剧烈波动。

4 附录：Python 源代码实现

本代码同时实现了 Lagrange、Newton 和分段线性插值，并将计算结果输出为数据以便于画图比对。

```
1 import math
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # 设置 matplotlib 支持中文显示
6 plt.rcParams['font.sans-serif'] = ['SimHei'] # 用于正常显示中文标签
7 plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题
8
9 # ----- 插值算法实现（与之前相同） -----
10 def R(x):
11     """原函数  $R(x) = 1/(1+x^2)$ """
12     return 1.0 / (1.0 + x * x)
13
14 def lagrange(x, xi, yi):
15     """拉格朗日插值"""
16     n = len(xi)
17     result = 0.0
18     for i in range(n):
19         term = yi[i]
20         for j in range(n):
21             if i != j:
22                 term *= (x - xi[j]) / (xi[i] - xi[j])
23         result += term
24     return result
25
26 def newton(x, xi, yi):
27     """牛顿插值（差商表+Horner）"""
28     n = len(xi)
29     f = [[0.0] * n for _ in range(n)]
30     for i in range(n):
31         f[i][0] = yi[i]
32     for j in range(1, n):
33         for i in range(j, n):
34             f[i][j] = (f[i][j-1] - f[i-1][j-1]) / (xi[i] - xi[i-j])
35     res = f[n-1][n-1]
36     for i in range(n-2, -1, -1):
37         res = f[i][i] + (x - xi[i]) * res
38     return res
39
40 def piecewise_linear(x, xi, yi):
41     """分段线性插值"""
42     n = len(xi)
43     if x <= xi[0]:
44         return yi[0]
45     if x >= xi[-1]:
```

```

46         return yi[-1]
47     for i in range(n-1):
48         if xi[i] <= x <= xi[i+1]:
49             return yi[i] + (yi[i+1] - yi[i]) * (x - xi[i]) / (xi[i+1] - xi[i])
50     return 0.0
51
52 # ----- 生成节点 -----
53 # 1. 切比雪夫节点 (用于20次Lagrange插值)
54 PI = math.acos(-1.0)
55 x_cheb = [5.0 * math.cos((2.0*i + 1.0) / 42.0 * PI) for i in range(21)]
56 y_cheb = [R(x) for x in x_cheb]
57
58 # 2. 等距节点 (用于Newton和分段线性插值)
59 x_eq = [-5.0 + i for i in range(11)]
60 y_eq = [R(x) for x in x_eq]
61
62 # ----- 生成绘图数据 -----
63 x_plot = np.linspace(-5, 5, 1000) # 1000个点用于平滑绘图
64 y_true = R(x_plot)
65 y_lagrange = [lagrange(x, x_cheb, y_cheb) for x in x_plot]
66 y_newton = [newton(x, x_eq, y_eq) for x in x_plot]
67 y_piecewise = [piecewise_linear(x, x_eq, y_eq) for x in x_plot]
68
69 # ----- 绘图 -----
70 plt.figure(figsize=(12, 7))
71
72 # 原函数
73 plt.plot(x_plot, y_true, 'k-', linewidth=2, label='原函数  $R(x)=1/(1+x^2)$ ')
74
75 # 20次Lagrange插值 (切比雪夫节点)
76 plt.plot(x_plot, y_lagrange, 'b--', linewidth=1.5, label='20次Lagrange插值 (切比雪夫节点)')
77
78 # 10次Newton插值 (等距节点, Runge现象)
79 plt.plot(x_plot, y_newton, 'r:', linewidth=1.5, label='10次Newton插值 (等距节点)')
80
81 # 分段线性插值
82 plt.plot(x_plot, y_piecewise, 'g-.', linewidth=1.5, label='分段线性插值')
83
84 # 标记节点 (可选, 增强可读性)
85 plt.plot(x_cheb, y_cheb, 'bo', markersize=3, label='切比雪夫节点')
86 plt.plot(x_eq, y_eq, 'rs', markersize=3, label='等距节点')
87
88 # 设置坐标轴范围 (突出Runge现象, 同时保证其他曲线可见)
89 plt.xlim(-5.5, 5.5)
90 plt.ylim(-0.5, 1.5) # 牛顿插值在边界处会剧烈震荡, 限制y轴范围以观察全貌
91
92 plt.xlabel('x', fontsize=12)

```

```

93 plt.ylabel('y', fontsize=12)
94 plt.title('三种插值方法与原函数  $R(x)$  的对比', fontsize=14)
95 plt.legend(loc='upper right', fontsize=10)
96 plt.grid(True, linestyle=':', alpha=0.6)
97
98 # 添加文字说明Runge现象和切比雪夫节点效果
99 plt.text(3.5, 1.3, 'Runge现象\n（等距节点高次震荡）', ha='center', fontsize=9,
100         bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.8))
101 plt.text(-4.5, 0.2, '切比雪夫节点\n抑制震荡', ha='center', fontsize=9,
102         bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.8))
103
104 # 保存图像（与LaTeX文档中引用的文件名一致）
105 plt.savefig('plot_results.png', dpi=300, bbox_inches='tight')
106 plt.show()
107
108 print("图像已保存为 plot_results.png")

```